

TokenButler System

企業 LLM API Token 與付費帳號管理系統

開發設計規劃說明書

SAI — System Analysis with AI

依循人機協作三大定律推動

版本 v1.0

發行日期:2026 年 5 月

目錄

1. 系統環境與開發架構
 - 1.1 系統名稱與定位
 - 1.2 技術棧
 - 1.3 Docker 容器化架構
 - 1.4 服務埠 (localhost:22000)
 - 1.5 資料庫位置與連線
2. 系統開發目的與主要功能
 - 2.1 開發背景
 - 2.2 系統開發目的
 - 2.3 主要功能總覽
 - 2.4 人機協作三大定律對應
3. 系統模組化功能說明
 - 3.1 模組化設計總綱
 - 3.2 M1 組織人員模組
 - 3.3 M2 資產盤點模組
 - 3.4 M3 財務支出模組
 - 3.5 M4 資源使用模組
 - 3.6 M5 使用人員與應用模組
 - 3.7 M6 申請流程模組
 - 3.8 M7 使用分析模組
 - 3.9 M8 Office 客服模組
4. 目錄結構與程式碼清單
 - 4.1 最上層目錄
 - 4.2 Backend 完整目錄
 - 4.3 Frontend 完整目錄
5. 各程式碼檔案職責說明
 - 5.1 Backend 程式碼說明
 - 5.2 Frontend 程式碼說明
6. 跨模組共用程式碼說明

6.1 Backend 跨模組共用

6.2 Frontend 跨模組共用

6.3 前後端契約對應

1. 系統環境與開發架構

1.1 系統名稱與定位

本系統正式名稱為 TokenButler System,中文定位為「企業 LLM API Token 與付費帳號管理系統」。命名隱喻取自「Token + Butler (管家)」,意指系統扮演企業內部 LLM 資產的專屬管家,協助 IT 與管理單位進行 API Key 與付費帳號的全生命週期管理。

系統定位為企業內部部署 (On-Premise) 之資產治理平台,涵蓋資產盤點、財務管控、使用追蹤、申請審核、效益分析、AI 客服等六大職能。

1.2 技術棧

本系統採前後端分離架構,技術選型如下表所列。

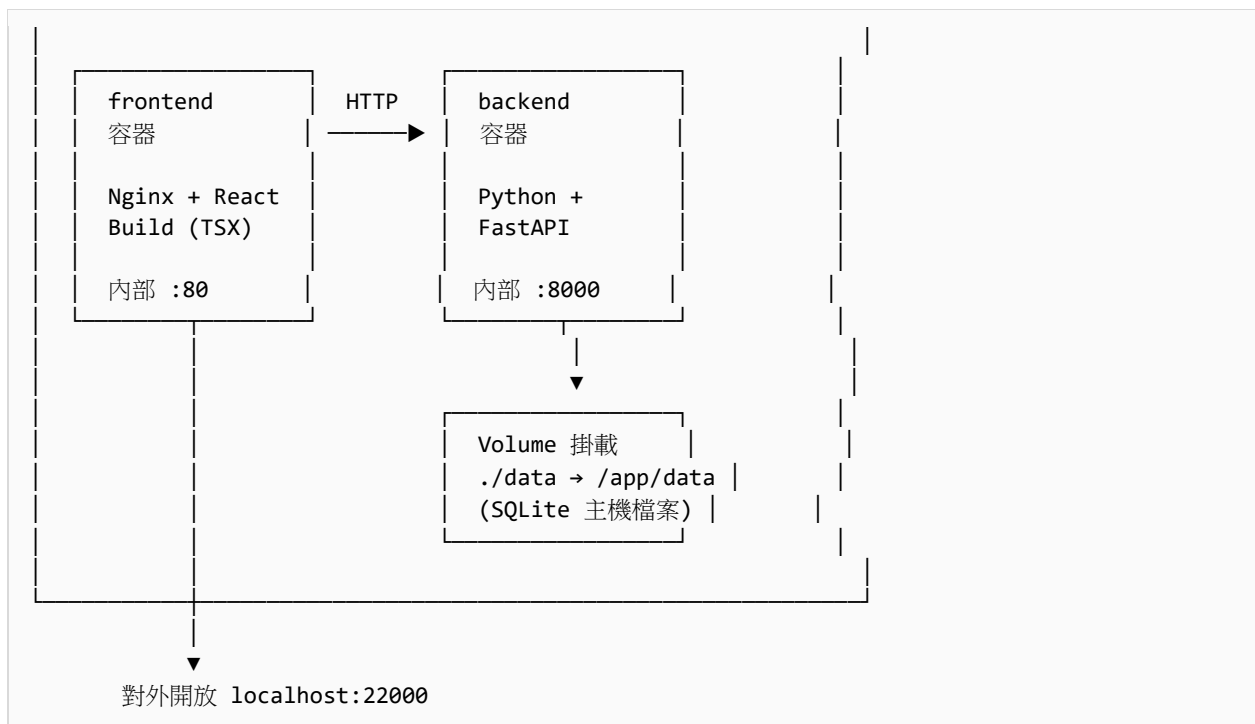
層級	技術	版本/說明
前端語言	TypeScript + React	TS 5.x / React 18.x,副檔名 .tsx (含 JSX) 與 .ts (純邏輯)
前端建構	Vite	5.x,內建 TypeScript 編譯支援
前端樣式	CSS	每個組件配對一個 .css 檔,不採 Tailwind
後端語言	Python	3.11
後端框架	FastAPI	0.110+,提供 async 與自動 OpenAPI 文件
後端 ORM	SQLAlchemy	2.x
資料庫	SQLite	3.x,單檔輕量資料庫
驗證	JWT + bcrypt	JSON Web Token 簽發、bcrypt 密碼雜湊
加密	AES-GCM	對 API Key、帳號密碼進行對稱加密
容器化	Docker + Compose	統一容器編排,單機部署

1.3 Docker 容器化架構

系統使用 Docker Compose 統一編排兩個服務容器:backend 與 frontend。SQLite 資料庫檔案透過 volume 掛載於主機,確保資料持久化且可在容器外進行備份。

容器拓撲如下:





1.3.1 容器服務說明

容器服務	映像基礎	對外埠	職責
frontend	node:20-alpine → nginx:alpine 多階段建構	22000	提供 React 靜態檔案,反向代理 /api 至 backend
backend	python:3.11-slim	(僅內部)	FastAPI 應用伺服器,連接 SQLite

1.3.2 啟動指令

於專案根目錄 TokenButler/ 執行以下指令啟動全系統:

```
# 第一次啟動 (建構映像並啟動)
docker-compose up --build -d

# 之後啟動
docker-compose up -d

# 查看日誌
docker-compose logs -f

# 停止
docker-compose down
```

1.4 服務埠 (localhost:22000)

系統對外僅開放單一服務埠:22000。使用者透過瀏覽器訪問 <http://localhost:22000> 即可進入登入頁面 (uid + admin 登入機制)。

Frontend 容器內的 Nginx 同時負責兩件事:(1) 提供 React 靜態檔案;(2) 將 /api 開頭的請求反向代理至 backend 容器內部的 8000 埠。Backend 不直接對外開放,降低攻擊面。

此架構同時解決瀏覽器 CORS 問題,因前後端對使用者而言為同一個來源 (Same Origin)。

1.5 資料庫位置與連線

SQLite 資料庫檔案 tokenbutler.db 存放於專案根目錄下的 data/ 資料夾中,透過 Docker Volume 掛載進 backend 容器。

項目	說明
主機路徑	/TokenButler/data/tokenbutler.db
容器內路徑	/app/data/tokenbutler.db
環境變數	DATABASE_URL=sqlite:///./data/tokenbutler.db
連線方式	SQLAlchemy engine 從 .env 讀取 DATABASE_URL 後建立連線池
初始化	首次啟動執行 backend/db/init_db.py,讀取 schema.sql 建表 + 載入 seed.sql
備份策略	直接複製 data/tokenbutler.db 即可完成完整備份

2. 系統開發目的與主要功能

2.1 開發背景

近年企業內部 LLM 應用迅速擴張,各部門紛紛申請 Claude、ChatGPT、Gemini 等 LLM 服務。然而企業層級對這些「Token 資產」普遍缺乏統一管控,衍生以下痛點:

- API Key 散落各員工手中,缺乏盤點機制,離職後形同遺失資產
- 付費訂閱 (Paid SaaS) 與 API 計量 (Paid API) 兩種計費模式混雜,財務難以彙整
- 速率限制 (例如 Claude Pro 5 小時/1M tokens) 缺乏監控,經常觸發限制影響業務
- 員工不知道公司有哪些可用 AI 資源,反覆申請或私下自費購買
- 無法量化各部門 AI 投資的回報率 (ROI),預算分配缺乏依據

2.2 系統開發目的

本系統開發目的為提供企業 LLM 資產的「人、組織、資產、使用」四維統一治理平台,具體目標如下:

目的編號	目的描述
G1	建立企業 LLM 資產 (API Key、付費帳號、Email 帳號) 單一真實來源 (Single Source of Truth)
G2	落實付費合約與實際支出的對應關係,支援月付/年付/用量計費三種模式
G3	即時監控 Token 速率限制與用量,預防業務中斷
G4	以員工角色為基礎建立資產分配與權限隔離機制
G5	建立 AI 資源申請的標準化流程,含主管審核與系統管理員核發
G6	提供統計分析與 ROI 計算,作為未來預算分配的依據
G7	提供 AI 客服協助員工查詢可用資源、引導申請流程

2.3 主要功能總覽

系統依職能分為以下八大主要功能 (對應八大模組):

編號	功能名稱	主要使用者
F1	組織人員管理 (員工、部門、角色)	admin
F2	資產盤點管理 (Email 帳號、LLM 模型、資產池)	admin、tech_user

編號	功能名稱	主要使用者
F3	財務支出管理 (合約、月付/年付、預警)	admin、manager、exec_manager
F4	資源使用管理 (用量日誌、速率限制監控)	admin、tech_user、user
F5	使用人員與應用管理 (資產分配、產出登錄)	admin、user
F6	申請流程管理 (申請、審核、核發)	user、manager、admin
F7	使用分析管理 (儀表板、成本、ROI、預測)	manager、exec_manager、admin
F8	Office 客服 (AI 對話、智能申請)	全體員工

2.4 人機協作三大定律對應

本系統開發遵循「人機協作三大定律」,該定律於系統各模組之具體落實如下:

定律	原則	系統具體落實
第一定律	協助而非取代	M8 AI 客服協助填寫申請單,但最終送出由人按下確認;M7 預測模型提出建議,但預算分配由主管決定
第二定律	可追溯可稽核	所有資產異動寫入 asst_account_history;所有申請寫入 aply_approvals;每筆 API 呼叫寫入 usge_logs
第三定律	人有最終否決權	AI 可建議分配方案,但 manager 與 admin 才能核准申請;AI 不可直接修改資產狀態

3. 系統模組化功能說明

3.1 模組化設計總綱

本系統採八大業務模組架構,每個模組由獨立的資料表前綴、後端 Python 套件、前端 React 模組三層構成,確保「DB → API → UI」一條垂直線完整貫穿。

模組編號採 M1 至 M8 編號,各層命名規則如下表:

模組碼	業務名稱	DB 表前綴	Backend 路徑	Frontend 路徑
M1	組織人員	orgs_*	modules/m1_orgs/	modules/m1_orgs/
M2	資產盤點	asst_*	modules/m2_asst/	modules/m2_asst/
M3	財務支出	fin*_*	modules/m3_fina/	modules/m3_fina/
M4	資源使用	usge_*	modules/m4_usge/	modules/m4_usge/
M5	使用人員與應用	pers_*, appl_*	modules/m5_pers/	modules/m5_pers/
M6	申請流程	aply_*	modules/m6_aply/	modules/m6_aply/
M7	使用分析	anly_*	modules/m7_anly/	modules/m7_anly/
M8	Office 客服	offc_*	modules/m8_offc/	modules/m8_offc/

3.2 M1 組織人員模組

3.2.1 模組職責

管理企業組織階層 (公司、事業群、部門) 與員工資料,定義五種角色 (admin、exec_manager、manager、tech_user、user) 並作為其他模組權限隔離的基礎。

3.2.2 資料表

表名	說明
orgs_users	員工主檔,含 uid 登入帳號、姓名、Email、角色、所屬部門
orgs_groups	部門/群組階層,支援透過 parent_id 建立樹狀結構

3.2.3 主要功能

- 新增、編輯、停用員工帳號
- 建立部門階層,指派部門主管

- 設定部門月度預算 (供 M3 財務模組使用)
- 提供其他模組進行權限判定 (admin 跨部門、manager 限本部門等)

3.3 M2 資產盤點模組

3.3.1 模組職責

管理企業 LLM 資產的三層結構:Email 帳號實體、LLM 模型字典、資產池 (Token Pool)。本模組是整個系統的核心資產資料來源。

3.3.2 三層綁定邏輯

關鍵業務邏輯為「每一個 LLM 模型必須綁一個專屬帳號 (Email)」。Email 帳號區分為三種擁有者類型:

擁有者類型	代碼	說明	風險
專屬虛擬帳號	dedicated	IT 為 AI 服務專門建立的信箱,如 ai-rd@company.com	最安全,推薦預設
員工個人帳號	employee	用某員工 Email 註冊,如 john@company.com	員工離職時需轉移
共用行政帳號	shared_admin	多人共用的行政信箱,如 it-admin@company.com	需強制 MFA 與授權清單

3.3.3 資料表

表名	說明
asst_models	LLM 模型字典 (供應商、模型名、速率限制、單價)
asst_accounts	Email 帳號實體,含擁有者類型、保管人、加密密碼
asst_account_access	共用帳號的授權清單
asst_account_history	帳號異動軌跡 (建立、轉移、輪換、退役)
asst_pools	Token 資產池,綁定 acct_id 與 modl_id,代表一次訂閱實例

3.3.4 主要功能

- Email 帳號 CRUD,含三種擁有者類型動態表單
- LLM 模型字典維護 (Anthropic、OpenAI、Google、Azure 等)
- 資產池管理:綁定帳號與模型,加密儲存 API Key

- 員工離職時的帳號轉移流程 (acct_status: active → transferring → active)
- 帳號生命週期狀態機 (active / locked / transferring / retired)

3.4 M3 財務支出模組

3.4.1 模組職責

管理資產池的合約計費資訊與實際支出明細,支援月付、年付、用量三種計費模式。提供合約到期、預算超標等財務預警。

3.4.2 資料表

表名	說明
fina_contracts	合約計費表,含 billing_cycle、月付/年付金額、合約期間
fina_spending	支出明細表,記錄每月實際支出與 token 用量
fina_alerts	財務預警 (合約到期、預算超標、異常支出)

3.4.3 月付/年付相依邏輯

關鍵業務邏輯:月付費與年付費為相依互斥關係,但需可推算「月攤提成本」用於跨資產比較。

- billing_cycle='monthly' → yearly_fee 為 NULL,monthly_amortized = monthly_fee
- billing_cycle='yearly' → monthly_fee 為 NULL,monthly_amortized = yearly_fee / 12
- billing_cycle='usage_based' → 兩者皆 NULL,支出走 fina_spending 累計

3.5 M4 資源使用模組

3.5.1 模組職責

追蹤每筆 LLM 呼叫的 token 用量,並維護資產池的即時速率限制狀態。本模組是支援所有分析與計費的最底層資料源。

3.5.2 資料表

表名	說明
usge_logs	使用日誌 (誰、何時、用了多少 token)
usge_limits_state	速率限制即時狀態 (5 小時/今日/本週/本月累計與限額)

3.5.3 SaaS 與 API 限制差異

資源型態	限制邏輯	範例
paid_saas	時間窗限制 (5 小時/週)	Claude Pro:每 5 小時 1M tokens
paid_api	總量單位成本	Claude API:USD 3 per 1M input tokens

3.6 M5 使用人員與應用模組

3.6.1 模組職責

管理資產池與員工之間的分配關係,以及員工使用 AI 的應用用途與產出登錄,作為 ROI 分析的資料來源。

3.6.2 資料表

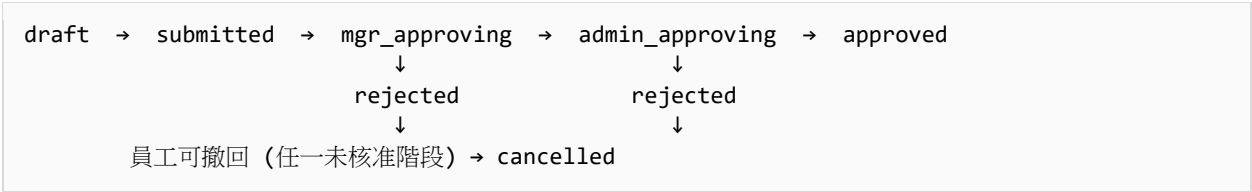
表名	說明
pers_assignments	資產分配關係 (誰可用哪個 pool、個人配額上限、起迄日)
appl_purposes	應用用途字典 (程式碼產生、文件撰寫、資料分析等)
appl_outputs	員工 AI 產出登錄 (含估算商業價值,供 ROI 計算)

3.7 M6 申請流程模組

3.7.1 模組職責

管理員工申請 AI 資源的標準化流程,含申請單、多關審核、自動分配。是員工取得 AI 資源的唯一正式管道。

3.7.2 申請狀態機



3.7.3 資料表

表名	說明
aply_requests	申請單主檔 (申請人、目標模型、理由、狀態、申請管道)

表名	說明
aply_approvals	審核軌跡 (誰、何時、批准或退回、意見)

3.8 M7 使用分析模組

3.8.1 模組職責

產出企業層級的使用統計、成本分析、ROI 計算、預測。提供可量化的 AI 投資依據。

3.8.2 資料表

表名	說明
anly_snapshots	統計快照 (按日/週/月、按使用者/部門/公司彙整)
anly_forecasts	預測表 (下月/下季/下年的成本與用量預測)

3.8.3 主要報表

- 儀表板:總成本、總用量、ROI 即時數據
- 成本分析:跨資產、跨部門、跨時間的支出明細
- ROI 分析:成本對應產出價值的比率
- 預測:基於歷史數據的下期預測
- 部門比較:各部門用量與效益對比

3.9 M8 Office 客服模組

3.9.1 模組職責

提供 AI 客服服務,協助員工查詢可用資源、引導建立申請單、回答 AI 工具使用問題。原規劃名稱為 Humatrix,現定名 Office 客服。

3.9.2 資料表

表名	說明
offc_conversations	對話主檔 (對話發起人、意圖、狀態、關聯申請單)
offc_messages	對話訊息明細 (角色、內容、消耗 token、使用哪個 pool 回應)

3.9.3 三大意圖

意圖代碼	說明	後續動作
ask_license	員工詢問是否有可用 AI 資源	查詢該員工的 pers_assignments 並列出
submit_request	員工要求申請新資源	引導填表,呼叫 M6 建立 aply_requests
general_qa	AI 工具一般問答	用 RAG 從 docs/ 查找答案

4. 目錄結構與程式碼清單

4.1 最上層目錄

專案根目錄 TokenButler/ 採前後端分離,並以 docker-compose.yml 統一編排。

```
TokenButler/
├── backend/                # FastAPI + SQLite 後端 (Python)
├── frontend/              # React + TSX 前端
├── data/                  # SQLite 資料庫檔案 (gitignore)
│   └── tokenbutler.db     # 主資料庫,由 init_db.py 自動建立
├── docker/               # Docker 設定檔
│   ├── backend.Dockerfile
│   ├── frontend.Dockerfile
│   └── nginx/
│       └── default.conf   # Nginx 反代設定
├── docs/                 # SAI 文件
│   ├── 01_sai_planning.docx # 本文件
│   ├── 02_db_schema.md
│   └── 03_api_spec.md
├── vault/               # 加密金鑰 (gitignore)
│   └── master.key        # AES master key
├── docker-compose.yml    # 統一啟動入口
├── .env.example          # 環境變數範本
├── .gitignore
└── README.md            # 含 localhost:22000 啟動指引
```

4.2 Backend 完整目錄

後端 backend/ 採八大模組化結構。每個模組固定包含 models.py、schemas.py、router.py、service.py 四個檔案。另預留 lava/、ccma/、llmafunction/ 三個空目錄供未來擴充。

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py           # FastAPI 入口,註冊所有模組路由
│   ├── database.py       # SQLAlchemy engine、Session、get_db
│   ├── security.py       # JWT、bcrypt 密碼、AES 加解密
│   ├── permissions.py    # 五種角色判定邏輯
│   ├── deps.py           # FastAPI 依賴注入 (current_user、db)
│   └── modules/
│       └── __init__.py   # 八大業務模組
│           ├── m1_orgs/
│               ├── __init__.py
│               ├── models.py
│               ├── schemas.py
│               └── router.py
│                   ├── orgs_users、orgs_groups ORM
│                   ├── Pydantic 結構
│                   └── /api/orgs/* 端點
```

└─ service.py	# 部門階層、員工 CRUD
└─ m2_asst/	# M2 資產盤點
└─ ── __init__.py	
└─ ── models.py	# 5 張表 ORM
(models/accounts/access/history/pools)	
└─ ── schemas.py	# /api/asst/* (accounts、models、pools)
└─ ── router.py	# 帳號生命週期、轉移、加密
└─ ── service.py	
└─ m3_fina/	# M3 財務支出
└─ ── __init__.py	
└─ ── models.py	# contracts、spending、alerts
└─ ── schemas.py	
└─ ── router.py	# /api/fina/*
└─ ── service.py	# 月付/年付攤提、預警
└─ m4_usge/	# M4 資源使用
└─ ── __init__.py	
└─ ── models.py	# usge_logs、usge_limits_state
└─ ── schemas.py	
└─ ── router.py	# /api/usge/*
└─ ── service.py	# 用量寫入、限制狀態計算
└─ m5_pers/	# M5 使用人員與應用
└─ ── __init__.py	
└─ ── models.py	# pers_assignments、appl_purposes、
appl_outputs	
└─ ── schemas.py	# /api/pers/*
└─ ── router.py	# 分配、產出登錄
└─ ── service.py	
└─ m6_aply/	# M6 申請流程
└─ ── __init__.py	
└─ ── models.py	# aply_requests、aply_approvals
└─ ── schemas.py	
└─ ── router.py	# /api/aply/*
└─ ── service.py	# 申請狀態機、審核流程
└─ m7_anly/	# M7 使用分析
└─ ── __init__.py	
└─ ── models.py	# anly_snapshots、anly_forecasts
└─ ── schemas.py	
└─ ── router.py	# /api/anly/*
└─ ── service.py	# 統計聚合、ROI 計算、預測
└─ m8_offc/	# M8 Office 客服
└─ ── __init__.py	
└─ ── models.py	# offc_conversations、offc_messages
└─ ── schemas.py	
└─ ── router.py	# /api/offc/*
└─ ── service.py	# AI 客服核心、意圖識別、LLM 呼叫
└─ auth/	# 登入登出 (跨模組,獨立放)
└─ ── __init__.py	
└─ ── router.py	# /api/auth/login、logout、refresh
└─ ── service.py	# JWT 簽發、登入驗證

├─ lava/	# ★ 預留 (未來擴充,目前空目錄)
│ └─ __init__.py	
├─ ccma/	# ★ 預留 (未來擴充,目前空目錄)
│ └─ __init__.py	
├─ llmafunction/	# ★ 預留 (未來擴充,目前空目錄)
│ └─ __init__.py	
├─ db/	# 資料庫初始化 (取代 Alembic)
│ ├── schema.sql	# 完整 CREATE TABLE 語法 (21 張表)
│ ├── seed.sql	# 初始資料 (admin 帳號、LLM 模型字典)
│ └─ init_db.py	# 執行 schema.sql + seed.sql 腳本
├─ requirements.txt	# Python 依賴
├─ Dockerfile	# 後端容器映像
└─ README.md	# 後端啟動指引

4.3 Frontend 完整目錄

前端 frontend/ 採與後端對應的八大模組化結構。每個模組僅包含 pages/ 與 components/ 兩層,所有檔案皆為 .tsx (含 JSX) 或 .ts (純邏輯),並配對 .css 樣式檔。

frontend/	
├─ public/	
│ ├── index.html	
│ ├── favicon.ico	
│ └─ tokenbutler-logo.svg	
├─ src/	
│ ├── main.tsx	# React 入口,掛載 App
│ ├── App.tsx	# 根組件,包裹 Provider
│ ├── router.tsx	# 集中註冊所有頁面路由
│ └─ api.ts	# axios 實例 + 統一錯誤處理
│ └─ shared/	# 跨模組共用
│ ├── types.ts	# 共用 interface (User、Pool 等)
│ ├── enums.ts	# 共用列舉 (角色、狀態)
│ ├── auth.tsx	# 登入態 Context + ProtectedRoute
│ ├── format.ts	# 金額/日期/token 格式化
│ └─ components/	# 共用 UI 組件
│ ├── Layout.tsx	
│ ├── Layout.css	
│ ├── Sidebar.tsx	# 側邊選單 (依角色顯示模組)
│ ├── Sidebar.css	
│ ├── Header.tsx	
│ ├── Header.css	
│ ├── Table.tsx	# 共用表格
│ ├── Table.css	
│ ├── Modal.tsx	
│ ├── Modal.css	
│ ├── Button.tsx	
│ └─ Button.css	

- Input.tsx - Input.css - Badge.tsx - Badge.css	
- styles/ - globals.css - theme.css	# 全域 reset + CSS 變數 # 顏色、字型主題
modules/	# 八大業務模組 (對應 backend)
m1_orgs/	# M1 組織人員
- pages/ - UsersManagePage.tsx - UsersManagePage.css - GroupsManagePage.tsx - GroupsManagePage.css - components/ - UserForm.tsx - UserForm.css - GroupTree.tsx - GroupTree.css	
m2_asst/	# M2 資產盤點
- pages/ - AccountsManagePage.tsx - AccountsManagePage.css - ModelsManagePage.tsx - ModelsManagePage.css - PoolsManagePage.tsx - PoolsManagePage.css - components/ - AccountForm.tsx - AccountForm.css - PoolForm.tsx - PoolForm.css	# Email 帳號管理 (核心頁) # 三類型動態表單
m3_fina/	# M3 財務支出
- pages/ - ContractsManagePage.tsx - ContractsManagePage.css - SpendingPage.tsx - SpendingPage.css - AlertsPage.tsx - AlertsPage.css - components/ - ContractForm.tsx - ContractForm.css - SpendingChart.tsx - SpendingChart.css	# 月付/年付互斥表單
m4_usge/	# M4 資源使用
- pages/ - UsageLogsPage.tsx - UsageLogsPage.css - LimitsMonitorPage.tsx - LimitsMonitorPage.css - components/ - UtilizationGauge.tsx	# 5h 速率窗儀表



```
|
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
├─ Dockerfile
└─ README.md

# TypeScript 設定
# Vite 設定 (port 22000)
```

5. 各程式碼檔案職責說明

5.1 Backend 程式碼說明

5.1.1 應用入口與核心檔案

檔案路徑	職責說明
app/main.py	FastAPI 應用入口。建立 app 實例、註冊 CORS 中介層、註冊全域例外處理器、透過 lifespan context 初始化資料庫,並掛載 auth/router 與八大模組的 router。提供 /health 健康檢查與 /api/docs Swagger 介面
app/database.py	資料庫連線管理。從 .env 讀取 DATABASE_URL,建立 SQLAlchemy engine 並啟用 SQLite WAL 模式,設定 PRAGMA foreign_keys=ON。提供 SessionLocal sessionmaker 與 get_db() 依賴注入函式
app/security.py	安全性核心。包含 JWT 簽發與驗證、bcrypt 密碼雜湊與比對、AES-GCM 對稱加解密 (用於 API Key 與帳號密碼)。從 vault/master.key 載入主金鑰
app/permissions.py	RBAC 角色判定。提供 require_role() 裝飾器、can_view_group() 部門資料隔離判定、can_approve_apply() 審核權限判定。判定邏輯:admin/exec_manager 全可,manager 限本部門及下屬,user/tech_user 限自己
app/deps.py	FastAPI 共用依賴。提供 get_current_user() 從 JWT 解出當前使用者、get_db() 資料庫 Session 注入。供所有 router 使用

5.1.2 M1 組織人員模組檔案

檔案路徑	職責說明
modules/m1_orgs/models.py	SQLAlchemy ORM 定義:orgs_users 表 (員工主檔含 uid、姓名、Email、密碼雜湊、角色、grps_id 部門外鍵)、orgs_groups 表 (部門/群組階層含 parent_id 自關聯)
modules/m1_orgs/schemas.py	Pydantic 結構:UserCreate、UserUpdate、UserResponse、GroupCreate、GroupTreeResponse 等請求與回應結構
modules/m1_orgs/router.py	/api/orgs/* 路由:GET/POST/PUT /users、GET/POST /groups、GET /groups/tree、GET /users/me 等端點
modules/m1_orgs/service.py	業務邏輯:create_user()、update_user_role()、build_group_tree()、get_user_chain()、calculate_dept_budget_usage()

5.1.3 M2 資產盤點模組檔案

檔案路徑	職責說明
modules/m2_asst/models.py	ORM 定義 5 張表:asst_models (LLM 模型字典)、asst_accounts (Email 帳號實體,含 owner_type)、asst_account_access (共用帳號授權)、asst_account_history (帳號異動軌跡)、asst_pools (Token 資產池,綁定 acct_id 與 modl_id)
modules/m2_asst/schemas.py	Pydantic 結構:AccountCreate (三類型動態欄位驗證)、AccountTransfer、PoolCreate、ModelCreate 等。包含 CHECK 約束驗證 (employee 類型必填 owner_user_id 等)
modules/m2_asst/router.py	/api/asst/* 路由:accounts CRUD、pools CRUD、models CRUD、accounts/{id}/transfer、accounts/{id}/rotate-password、accounts/{id}/history、accounts/{id}/access
modules/m2_asst/service.py	核心業務:create_account() 三類型動態建立、transfer_account() 員工離職轉移、rotate_password() 密碼輪換、grant_access() 共用帳號授權、encrypt_credential() 與 decrypt_credential() 憑證加解密。所有寫操作觸發 account_history 軌跡記錄

5.1.4 M3 財務支出模組檔案

檔案路徑	職責說明
modules/m3_fina/models.py	ORM:fina_contracts (合約計費)、fina_spending (支出明細)、fina_alerts (預警)。contracts 表含 generated column monthly_amortized = monthly_fee or yearly_fee/12
modules/m3_fina/schemas.py	Pydantic:ContractCreate (含月付/年付互斥驗證)、SpendingResponse、AlertResponse
modules/m3_fina/router.py	/api/fina/* 路由:contracts CRUD、spending 查詢與彙整、alerts 列表與已讀標記
modules/m3_fina/service.py	業務邏輯:calculate_monthly_amortized() 月攤提計算、check_due_dates() 合約到期檢查、check_budget_overrun() 預算超標判定、create_alert() 預警產生

5.1.5 M4 資源使用模組檔案

檔案路徑	職責說明
modules/m4_usge/models.py	ORM:usge_logs (使用日誌,含 tokens_input、tokens_output、

檔案路徑	職責說明
	cost、timestamp)、usge_limits_state (速率限制即時狀態快取,各時間窗累計與閾值)
modules/m4_usge/schemas.py	Pydantic:UsageLogCreate、UsageLogQuery、LimitsStateResponse
modules/m4_usge/router.py	/api/usge/* 路由:logs 查詢 (依 pool/user 篩選)、limits 即時狀態查詢、limits 手動觸發重算
modules/m4_usge/service.py	業務邏輯:record_usage() 寫入用量並觸發限制狀態更新、recompute_limits_state() 聚合過去 5h/今日/本週/本月用量、check_rate_limit() 判斷是否觸發 80% 預警

5.1.6 M5 使用人員與應用模組檔案

檔案路徑	職責說明
modules/m5_pers/models.py	ORM:pers_assignments (資產分配關係)、appl_purposes (用途字典)、appl_outputs (產出登錄含估算商業價值)
modules/m5_pers/schemas.py	Pydantic:AssignmentCreate、MyAssetsResponse、OutputCreate、PurposeResponse
modules/m5_pers/router.py	/api/pers/* 路由:assignments CRUD、my-assets 查詢 (當前使用者可用資產)、outputs CRUD、purposes 字典
modules/m5_pers/service.py	業務邏輯:assign_pool_to_user() 分配資產、get_user_active_assets() 取得使用者可用資產、record_output() 產出登錄、calculate_user_quota_remaining() 個人額度計算

5.1.7 M6 申請流程模組檔案

檔案路徑	職責說明
modules/m6_aply/models.py	ORM:aply_requests (申請單主檔,含 status 狀態機欄位)、aply_approvals (審核軌跡)
modules/m6_aply/schemas.py	Pydantic:RequestCreate、RequestUpdate、ApprovalAction、RequestDetailResponse (含審核軌跡)
modules/m6_aply/router.py	/api/aply/* 路由:requests CRUD、requests/{id}/submit、requests/{id}/approve、requests/{id}/reject、requests/{id}/cancel、approval-queue (待審核佇列)

檔案路徑	職責說明
modules/m6_aply/service.py	核心狀態機邏輯:submit_request()、forward_to_manager()、approve_step() 多關審核、on_approved() 觸發自動分配 (寫入 M5 pers_assignments)、cancel_request()。包含狀態轉換守衛 (誰可以執行哪個轉換)

5.1.8 M7 使用分析模組檔案

檔案路徑	職責說明
modules/m7_anly/models.py	ORM:anly_snapshots (統計快照)、anly_forecasts (預測表)
modules/m7_anly/schemas.py	Pydantic:DashboardResponse、CostReportQuery、RoiReportResponse、ForecastResponse
modules/m7_anly/router.py	/api/anly/* 路由:dashboard 儀表板、cost 成本分析、usage 用量分析、roi ROI 分析、forecast 預測、dept-compare 部門比較
modules/m7_anly/service.py	業務邏輯:build_dashboard() 即時儀表板、aggregate_cost() 跨期跨部門成本彙整、calculate_roi() ROI 計算 (產出價值/成本)、forecast_next_period() 預測 (移動平均或線性回歸)

5.1.9 M8 Office 客服模組檔案

檔案路徑	職責說明
modules/m8_offc/models.py	ORM:offc_conversations (對話主檔,含 intent 意圖、conv_aply_id 關聯申請單)、offc_messages (訊息明細)
modules/m8_offc/schemas.py	Pydantic:ChatRequest、ChatResponse、ConversationResponse、MessageResponse
modules/m8_offc/router.py	/api/offc/* 路由:chat 對話入口、conversations 歷史對話、intent 意圖解析
modules/m8_offc/service.py	AI 客服核心邏輯:分類意圖 (ask_license/submit_request/general_qa)、查詢使用者可用資產 (呼叫 M5)、引導建立申請單 (呼叫 M6)、呼叫 LLM 提供回應。維護對話紀錄,每次呼叫記錄到 usge_logs (AI 客服本身也算用量)

5.1.10 認證模組 auth/ 檔案

檔案路徑	職責說明
auth/router.py	/api/auth/* 路由:login (uid + 密碼)、logout、refresh (Token 刷新)
auth/service.py	登入驗證邏輯:verify_credentials() 驗證 uid 與 bcrypt 密碼、issue_jwt() 簽發 JWT、refresh_token()。失敗計數與帳號鎖定機制

5.1.11 db/ 資料庫初始化

檔案路徑	職責說明
db/schema.sql	完整 CREATE TABLE 語法,涵蓋八大模組共 21 張表的 DDL,含主鍵、外鍵、UNIQUE 約束、CHECK 約束、索引
db/seed.sql	初始資料 INSERT:預設 admin 帳號 (uid=admin、密碼預設值)、LLM 模型字典 (Anthropic、OpenAI、Google 等供應商)、用途字典 (code_gen、doc_writing 等)
db/init_db.py	Python 腳本:檢查 data/tokenbutler.db 是否存在,若否則建立並依序執行 schema.sql 與 seed.sql。首次部署或測試環境用

5.1.12 預留目錄 (空)

目錄路徑	預留用途
app/lava/	預留 (Latent Value Assistant 之未來模組命名),目前僅放 __init__.py 空檔。預期擴充為「資產價值評估子模組」
app/ccma/	預留 (Cross-module Coordination Manager Assistant 之未來模組命名),目前僅放 __init__.py。預期擴充為「跨模組協調管理器」
app/llmafunction/	預留 (LLM-Augmented Function 之未來模組命名),目前僅放 __init__.py。預期擴充為「LLM 增強功能子模組」,封裝可被各模組複用的 LLM 工具函式

5.2 Frontend 程式碼說明

5.2.1 應用入口與根組件

檔案路徑	職責說明
src/main.tsx	React 應用入口。呼叫 createRoot 將 App 組件掛載至 #root DOM 節點。引入 globals.css 全域樣式
src/App.tsx	根組件。包裹 AuthProvider 認證 Context、QueryClient (TanStack Query) 資料層、Toast Provider 通知系統,渲染 Router

檔案路徑	職責說明
src/router.tsx	集中路由表。使用 react-router-dom v6 註冊所有頁面路由 (LoginPage、HomePage、八大模組頁面)。透過 ProtectedRoute 包裝需登入或特定角色的路由
src/api.ts	Axios 統一客戶端。建立 instance 設定 baseUrl=/api、Request 攔截器自動帶 Authorization Bearer Token、Response 攔截器 401 自動清空登入態並導向登入頁、統一觸發 Toast 顯示錯誤

5.2.2 共用基礎設施 shared/

檔案路徑	職責說明
shared/types.ts	跨模組共用 TypeScript interface:User、Group、Pool、Account、Pagination<T>、ApiResponse<T>、ErrorResponse。供各模組頁面直接 import 使用,確保前後端契約一致
shared/enums.ts	共用列舉常數:UserRole (admin/exec_manager/manager/tech_user/user)、AcctOwnerType (dedicated/employee/shared_admin)、AcctStatus (active/locked/transferring/retired)、AplyStatus (draft/submitted/.../approved)、PoolStatus、ModlType (paid_saas/paid_api)、BillingCycle
shared/auth.tsx	認證 Context 與 Hook。提供 AuthProvider、useAuth() Hook (回傳當前使用者、login()、logout())、ProtectedRoute 組件 (檢查登入態與角色,未通過則導向登入頁或 403 頁)
shared/format.ts	純函式工具:formatCurrency() 金額格式化、formatDate() 日期格式化、formatTokenCount() Token 數縮寫 (1.2M)、formatPercent() 百分比、formatDuration() 時間間隔

5.2.3 共用 UI 組件 shared/components/

檔案路徑	職責說明
Layout.tsx + .css	主框架:渲染 Header + Sidebar + 內容區 (Outlet)。所有登入後頁面共用此框架
Sidebar.tsx + .css	側邊選單:依當前使用者角色動態渲染模組選單 (admin 全部、manager 限部門、user 限個人)。使用 NavLink 標示當前路由
Header.tsx + .css	頂部導覽:顯示登入人、未讀預警 badge、登出按鈕、語言切換

檔案路徑	職責說明
Table.tsx + .css	泛型共用表格:支援分頁、排序、篩選、列點擊展開。使用 TypeScript 泛型 Table<T> 確保資料結構型別正確
Modal.tsx + .css	對話框元件:支援標題、內容、確認/取消按鈕、ESC 關閉
Button.tsx + .css	按鈕元件:支援 primary/secondary/danger 三種樣式、loading 狀態、disabled 狀態
Input.tsx + .css	表單輸入:含 label、錯誤訊息、required 標記。整合 react-hook-form 表單庫
Badge.tsx + .css	狀態徽章:依不同狀態顯示對應顏色 (active 綠、warning 橘、danger 紅)。共用於各模組狀態顯示

5.2.4 各模組 pages/ 與 components/

以下逐一說明八大模組所有檔案的職責。

M1 組織人員模組

檔案路徑	職責說明
m1_orgs/pages/UsersManagePage.tsx	員工管理頁面:渲染員工列表 (Table)、新增/編輯員工 Modal、停用員工確認對話框。呼叫 /api/orgs/users 系列端點
m1_orgs/pages/UsersManagePage.css	員工管理頁面樣式:表格欄位寬度、徽章配色、操作按鈕間距
m1_orgs/pages/GroupsManagePage.tsx	部門階層管理頁面:渲染樹狀結構、新增/編輯部門、設定部門主管與月度預算
m1_orgs/pages/GroupsManagePage.css	部門管理頁樣式:樹狀結構縮排、節點展開收合動畫
m1_orgs/components/UserForm.tsx	員工表單組件:含 uid、姓名、Email、密碼、角色 (下拉)、部門 (下拉)。表單驗證與送出
m1_orgs/components/UserForm.css	表單樣式:欄位間距、錯誤訊息顏色
m1_orgs/components/GroupTree.tsx	部門樹狀組件:遞迴渲染 group_parent_id 階層,支援展開收合、選取節點
m1_orgs/components/GroupTree.css	樹狀組件樣式:連接線、節點 hover 效果

M2 資產盤點模組

檔案路徑	職責說明
m2_asst/pages/AccountsManagePage.tsx	Email 帳號管理頁 (核心):渲染帳號列表並支援按擁有者類型、狀態、部門篩選。點擊列展開 Drawer 顯示 pools、共用授權清單、異動軌跡。提供新增、轉移、輪換密碼、退役等操作
m2_asst/pages/AccountsManagePage.css	帳號管理頁樣式:篩選列、列展開動畫、Drawer 寬度
m2_asst/pages/ModelsManagePage.tsx	LLM 模型字典管理:渲染各供應商模型清單,可新增、編輯、停用模型,設定速率限制與單價
m2_asst/pages/ModelsManagePage.css	模型管理頁樣式
m2_asst/pages/PoolsManagePage.tsx	資產池管理:顯示所有 token pool 列表,綁定帳號與模型,管理 API Key 與 status
m2_asst/pages/PoolsManagePage.css	資產池頁樣式
m2_asst/components/AccountForm.tsx	Email 帳號三類型動態表單:選擇 owner_type 後動態切換欄位 (dedicated 顯示部門、employee 顯示員工、shared_admin 顯示部門+強制 MFA+授權清單)
m2_asst/components/AccountForm.css	表單樣式:動態切換動畫、警示提示
m2_asst/components/PoolForm.tsx	資產池表單:選擇 acct_id 與 modl_id,設定總額度、註冊資訊、到期日,加密填入 API Key
m2_asst/components/PoolForm.css	資產池表單樣式

M3 財務支出模組

檔案路徑	職責說明
m3_fina/pages/ContractsManagePage.tsx	合約管理頁:列出所有資產的合約,顯示計費方式 (月付/年付/用量)、到期日。即將到期者高亮顯示
m3_fina/pages/ContractsManagePage.css	合約頁樣式
m3_fina/pages/SpendingPage.tsx	支出明細頁:依月份/部門/資產池彙整支出,並顯示 SpendingChart 折線圖
m3_fina/pages/SpendingPage.css	支出頁樣式
m3_fina/pages/AlertsPage.tsx	預警中心:列出所有財務預警 (合約到期、預算超標、異常支出),可標記已讀
m3_fina/pages/AlertsPage.css	預警頁樣式

檔案路徑	職責說明
m3_fina/components/ContractForm.tsx	合約表單:選擇 billing_cycle 後動態顯示月付/年付欄位 (互斥),計算月攤提預覽
m3_fina/components/ContractForm.css	合約表單樣式
m3_fina/components/SpendingChart.tsx	支出折線圖:使用 Recharts 渲染近 12 個月支出趨勢,支援切換維度 (按部門/按資產/按 LLM 供應商)
m3_fina/components/SpendingChart.css	圖表容器樣式

M4 資源使用模組

檔案路徑	職責說明
m4_usge/pages/UsageLogsPage.tsx	使用日誌頁:可篩選依使用者、資產池、時間範圍查詢底層 token 用量明細,支援匯出 CSV
m4_usge/pages/UsageLogsPage.css	日誌頁樣式
m4_usge/pages/LimitsMonitorPage.tsx	速率限制監控頁:顯示所有 paid_saas 類型資產池的 UtilizationGauge 即時儀表,警示超過 80% 使用率者
m4_usge/pages/LimitsMonitorPage.css	監控頁樣式:儀表板網格
m4_usge/components/UtilizationGauge.tsx	5h 速率窗使用率儀表:半圓 Gauge 圖,顯示 5h/今日/本月使用率,動態顏色 (綠/黃/紅)。每 60 秒輪詢更新
m4_usge/components/UtilizationGauge.css	儀表樣式

M5 使用人員與應用模組

檔案路徑	職責說明
m5_pers/pages/AssignmentsManagePage.tsx	分配關係管理頁 (admin 視角):顯示所有 pool-to-user 分配關係,可新增、撤銷分配,設定個人配額
m5_pers/pages/AssignmentsManagePage.css	分配管理頁樣式
m5_pers/pages/MyAssetsPage.tsx	我的資產頁 (一般使用者視角):顯示當前使用者可使用的所有資產池,含剩餘配額、速率狀態、到期日
m5_pers/pages/MyAssetsPage.css	我的資產頁樣式:卡片網格
m5_pers/pages/MyOutputsPage.tsx	我的產出登錄頁:列出個人 AI 產出記錄,可新增、編輯,填寫估算商業價值

檔案路徑	職責說明
m5_pers/pages/MyOutputsPage.css	產出頁樣式
m5_pers/components/AssignmentForm.tsx	分配表單:選擇 pool 與 user,設定配額上限、起迄日
m5_pers/components/AssignmentForm.css	分配表單樣式
m5_pers/components/OutputForm.tsx	產出登錄表單:標題、描述、用途分類(下拉)、消耗 token、估算商業價值
m5_pers/components/OutputForm.css	產出表單樣式

M6 申請流程模組

檔案路徑	職責說明
m6_aply/pages/NewRequestPage.tsx	新申請頁:員工填寫申請單(目標模型、理由、預期用量、申請天數),送出後進入審核佇列
m6_aply/pages/NewRequestPage.css	新申請頁樣式
m6_aply/pages/MyRequestsPage.tsx	我的申請紀錄頁:列出個人所有申請單與當前狀態,可撤回未核准的申請
m6_aply/pages/MyRequestsPage.css	我的申請頁樣式
m6_aply/pages/ApprovalQueuePage.tsx	待審核佇列頁(manager/admin 視角):列出待審核的申請單,可批准/退回/留言
m6_aply/pages/ApprovalQueuePage.css	審核佇列樣式
m6_aply/components/RequestForm.tsx	申請表單組件:含目標 LLM 模型(下拉)、是否指定 pool、申請理由、預期月用量、申請天數
m6_aply/components/RequestForm.css	申請表單樣式
m6_aply/components/AplyStatusFlow.tsx	申請狀態流程視覺化:用步驟條(Stepper)顯示 draft → submitted → mgr_approving → admin_approving → approved 的進度。標示當前步驟與審核者
m6_aply/components/AplyStatusFlow.css	狀態流程樣式:步驟條配色、連接線

M7 使用分析模組

檔案路徑	職責說明
m7_anly/pages/DashboardPage.tsx	分析儀表板:渲染 KpiCard 多張(總成本、總用量、總申請數、

檔案路徑	職責說明
	ROI),搭配 CostChart 趨勢圖
m7_anly/pages/DashboardPage.css	儀表板樣式:卡片網格、響應式佈局
m7_anly/pages/CostReportPage.tsx	成本分析頁:可選擇時間範圍、部門、資產池進行交叉分析,輸出表格與折線圖
m7_anly/pages/CostReportPage.css	成本分析頁樣式
m7_anly/pages/RoiReportPage.tsx	ROI 分析頁:計算各部門/各使用者的投資回報率 (產出價值/成本),散點圖顯示成本與價值的關係
m7_anly/pages/RoiReportPage.css	ROI 頁樣式
m7_anly/components/KpiCard.tsx	KPI 卡片組件:顯示一個關鍵指標含標題、數值、變化率 (上漲/下跌箭頭)
m7_anly/components/KpiCard.css	KPI 卡片樣式
m7_anly/components/CostChart.tsx	成本趨勢圖:Recharts 折線圖,支援切換時間粒度與維度
m7_anly/components/CostChart.css	圖表樣式

M8 Office 客服模組

檔案路徑	職責說明
m8_offc/pages/ButlerChatPage.tsx	AI 客服對話頁:渲染對話訊息流 (ChatBubble 多筆) 與 ChatInput 輸入區。意圖識別後動態渲染輔助卡片 (查詢資產、引導申請)
m8_offc/pages/ButlerChatPage.css	對話頁樣式:訊息氣泡配色、滾動行為
m8_offc/components/ChatBubble.tsx	對話氣泡組件:依角色 (user/assistant) 顯示不同樣式。支援 markdown 渲染、複製按鈕
m8_offc/components/ChatBubble.css	氣泡樣式:左右對齊、配色
m8_offc/components/ChatInput.tsx	對話輸入組件:文字輸入框 + 送出按鈕 + (預留) 語音按鈕。Enter 送出、Shift+Enter 換行
m8_offc/components/ChatInput.css	輸入區樣式

5.2.5 跨模組頁面 src/pages/

檔案路徑	職責說明
pages/LoginPage.tsx	登入頁:uid + 密碼表單,呼叫 /api/auth/login,成功後儲存 JWT 並導向 HomePage
pages/LoginPage.css	登入頁樣式:置中卡片、品牌 logo
pages/HomePage.tsx	登入後首頁:依當前使用者角色顯示對應的「快速進入」按鈕 (admin 進入管理模組、user 進入我的資產)
pages/HomePage.css	首頁樣式

5.2.6 設定檔說明

檔案路徑	職責說明
package.json	npm 依賴清單:react、react-dom、react-router-dom、axios、@tanstack/react-query、react-hook-form、recharts、zustand、typescript
tsconfig.json	TypeScript 編譯設定:target=ES2020、strict=true、jsx=react-jsx、設定路徑別名 @/* 對應 src/*
vite.config.ts	Vite 設定:server.port=22000 (開發用)、build.outDir=dist、proxy /api 至後端 8000 埠 (開發環境跨容器代理)
Dockerfile	多階段建構:Stage 1 用 node:20-alpine 執行 npm run build 產生靜態檔案;Stage 2 用 nginx:alpine 提供靜態檔案與 /api 反代

6. 跨模組共用程式碼說明

本章專門說明「不屬於任一模組,但被多個模組共用」的程式碼。理解這部分是維護本系統的關鍵,因為這些檔案修改會影響全系統行為。

6.1 Backend 跨模組共用

6.1.1 app/main.py — 系統總入口

此檔是整個 backend 的中樞,所有模組都透過此檔被掛載。具體職責:

- 建立 FastAPI app 實例 (含 title、version)
- 註冊 CORS 中介層 (允許 localhost:22000 來源)
- 註冊全域例外處理器 (將 ValueError、PermissionError 轉為標準 JSON 回應)
- 透過 lifespan context manager 在啟動時呼叫 init_db() 確保資料庫已建立
- 掛載 auth router (/api/auth) 與八大模組 router (/api/orgs、/api/asst、/api/fina、/api/usge、/api/pers、/api/aply、/api/anly、/api/offc)
- 提供 /health 健康檢查端點與 /api/docs Swagger UI

每新增一個模組,只需在此檔加一行 app.include_router() 即可掛載。

6.1.2 app/database.py — 資料庫連線管理

所有模組透過 get_db() 取得資料庫 Session。職責:

- 從 .env 讀取 DATABASE_URL=sqlite:///./data/tokenbutler.db
- 建立 SQLAlchemy engine 並設定 SQLite 特有 PRAGMA (foreign_keys=ON、journal_mode=WAL)
- 提供 Base = declarative_base() 給所有模組 ORM 繼承
- 提供 SessionLocal sessionmaker 與 get_db() generator function

被誰使用:所有模組的 router.py 都會 from app.database import get_db,用 Depends(get_db) 注入。

6.1.3 app/security.py — 安全核心

安全相關共用函式,被認證模組與資產模組大量使用。職責:

- JWT 簽發與驗證:create_access_token()、decode_token(),被 auth/service.py 與 deps.py 使用

- bcrypt 密碼雜湊:hash_password()、verify_password(),被 m1_orgs/service.py 建立員工時使用
- AES-GCM 加解密:encrypt()、decrypt(),被 m2_asst/service.py 在儲存 API Key 與帳號密碼時使用。從 vault/master.key 讀取主金鑰
- master key 輪換函式:rotate_master_key(),解密所有現有密文並用新金鑰重新加密

6.1.4 app/permissions.py — RBAC 權限判定

所有模組共用的權限邏輯,確保資料隔離一致性。職責:

- require_role(*roles) 裝飾器:作為 FastAPI Depends 使用,自動檢查 current_user 是否符合角色
- can_view_group(user, target_grps_id):部門資料隔離核心邏輯。
admin/exec_manager 全可、manager 限本部門與下層 (透過 orgs_groups.parent_id 遞迴查詢)、user/tech_user 限自己
- can_approve_aply(user, aply_request):審核權限判定,被 M6 申請流程模組使用

被誰使用:所有 router 中需要權限檢查的端點,都會 import 這些函式。修改本檔需特別小心,影響整個系統的權限模型。

6.1.5 app/deps.py — FastAPI 依賴注入

提供 FastAPI 端點共用的依賴函式:

- get_db():資料庫 Session 注入
- get_current_user(token = Depends(oauth2_scheme)):從 JWT 解出當前使用者,被所有需登入的端點使用
- get_current_active_user():get_current_user() + 確認 user_is_active=True

6.1.6 auth/ — 認證模組 (跨模組屬性)

認證雖然在程式碼上獨立成資料夾,但功能上跨所有模組使用。職責:

- auth/router.py:提供 /api/auth/login、logout、refresh 端點
- auth/service.py:驗證 uid 密碼、產生 JWT、刷新 Token、失敗計數鎖定機制

與 M1 模組的關係:auth 讀取 orgs_users 表的 uid 與密碼雜湊進行驗證,但本身不持有 ORM (引用 M1 的 models)。

6.1.7 db/init_db.py — 資料庫初始化

跨模組的資料庫建立腳本:

- 檢查 data/tokenbutler.db 是否存在,若否則建立

- 讀取 db/schema.sql 執行所有 CREATE TABLE (涵蓋八大模組 21 張表)
- 讀取 db/seed.sql 載入初始資料 (admin 帳號、LLM 模型字典、用途字典)
- 被 main.py 的 lifespan 在啟動時自動呼叫一次

6.2 Frontend 跨模組共用

6.2.1 src/api.ts — Axios 統一客戶端

所有模組頁面呼叫後端 API 都透過此檔的 axios instance。職責:

- 建立 axios instance 設定 baseUrl='/api'
- Request 攔截器:自動從 localStorage 讀取 JWT 並加在 Authorization header
- Response 攔截器:401 自動清空登入態並導向 /login,500 等錯誤統一觸發 Toast
- export 各 HTTP 方法 (get、post、put、delete) 供各模組使用

修改本檔會影響全系統 API 行為,例如更改認證方式、添加全域 header、修改錯誤處理。

6.2.2 src/shared/types.ts — 跨模組型別

定義所有模組共用的 TypeScript interface:

- User、Group、Account、Pool 等核心實體型別 (對應後端 ORM 模型)
- Pagination<T>:泛型分頁回應結構 { items: T[], total, page, page_size }
- ApiResponse<T>:統一 API 回應 wrapper
- ErrorResponse:標準錯誤回應結構

被誰使用:幾乎所有 .tsx 與 .ts 檔案。修改本檔可能影響大量檔案,需要 tsc --noEmit 全專案型別檢查。

6.2.3 src/shared/enums.ts — 共用列舉

定義跨模組使用的列舉常數型別:

- UserRole = 'admin' | 'exec_manager' | 'manager' | 'tech_user' | 'user'
- AcctOwnerType = 'dedicated' | 'employee' | 'shared_admin'
- AcctStatus = 'active' | 'locked' | 'transferring' | 'retired'
- AplyStatus = 'draft' | 'submitted' | 'mgr_approving' | 'admin_approving' | 'approved' | 'rejected' | 'cancelled'
- PoolStatus、ModlType、BillingCycle 等其他狀態列舉

關鍵原則:這些 enum 值必須與後端 Python 的字串值完全一致 (例如後端

m6_aply/models.py 中 status='submitted')。任何修改需前後端同步。

6.2.4 src/shared/auth.tsx — 認證 Context

認證狀態與權限判斷的核心。職責：

- AuthProvider:包裹整個應用,提供 currentUser 狀態
- useAuth():Hook,任何組件可呼叫取得 currentUser、login()、logout()
- ProtectedRoute:路由守衛組件,檢查登入態與所需角色,未通過則導向登入頁或 403 頁
- 使用 zustand 或 React Context 管理狀態,持久化至 localStorage

被誰使用:router.tsx 包裝所有受保護路由、Header.tsx 顯示登入入、Sidebar.tsx 依角色顯示選單。

6.2.5 src/shared/format.ts — 格式化工具

純函式工具,所有模組頁面共用:

- formatCurrency(amount, currency='USD'):金額格式化 USD 1,234.56
- formatDate(date, format='YYYY-MM-DD'):日期格式化
- formatTokenCount(n):Token 數縮寫 (1234567 → 1.23M)
- formatPercent(value, decimals=1):百分比格式化
- formatDuration(seconds):時間間隔人類可讀 (3600 → '1 hour')

6.2.6 src/shared/components/ — 共用 UI 組件

以下組件被多模組共用,修改會影響整體 UI 體驗:

組件	被誰使用	修改影響
Layout.tsx	所有登入後頁面	影響整體版面
Sidebar.tsx	Layout.tsx	影響選單顯示邏輯
Header.tsx	Layout.tsx	影響頂部資訊
Table.tsx	幾乎所有列表頁面 (M1~M8)	影響表格行為一致性
Modal.tsx	所有需確認的操作	影響對話框行為
Button.tsx	所有按鈕	影響全 UI 互動
Input.tsx	所有表單	影響全表單體驗
Badge.tsx	所有狀態顯示	影響狀態識別性

6.2.7 src/router.tsx — 集中路由

所有頁面路由集中在此檔註冊,而非分散在各模組。職責:

- 使用 react-router-dom v6 的 createBrowserRouter 建立路由樹
- 根路由 / 包裹 Layout,所有受保護頁面為子路由
- 從各模組 import 頁面組件 (例如 import { AccountsManagePage } from './modules/m2_asst/pages/AccountsManagePage')
- 透過 ProtectedRoute 包裝指定路由,標註所需角色

修改本檔常見場景:新增頁面、修改頁面路徑、調整路由權限。

6.3 前後端契約對應

前後端共用的「契約」必須兩邊同步,否則 API 呼叫會失敗。以下是必須對應的部分:

契約類型	後端位置	前端位置	同步機制
列舉值	shared/enums.py 或各模組 models.py CHECK 約束	src/shared/enums.ts	手動同步,修改需兩邊同改
API 端點路徑	各模組 router.py 的 @router.get('/path')	src/api.ts 與各頁面的 axios 呼叫	手動同步
回應結構	各模組 schemas.py 的 Pydantic 模型	src/shared/types.ts 與各模組型別	手動同步,可用 openapi-typescript 自動產生
錯誤碼	app/exceptions.py 自訂例外	src/api.ts 的攔截器	手動同步
JWT 載荷結構	app/security.py 的 create_access_token	src/shared/auth.tsx 解析 payload	手動同步

6.3.1 建議的同步流程

為了降低前後端契約不一致風險,建議遵循以下開發流程:

- 修改 API 端點或回應結構時,先改後端 schemas.py,再改前端 types.ts
- 新增列舉值時,先改後端 models.py CHECK 約束,再改前端 enums.ts
- 可考慮使用 npm run gen-types 自動從後端 /openapi.json 產生前端型別 (進階)
- 變更欄位名時,優先用 Git 全專案搜尋確認所有引用點

6.3.2 共用業務邏輯所在位置

由於業務邏輯散落在前後端各處,為了維護方便,以下整理跨模組業務邏輯的「真實來源」:

業務邏輯	真實來源 (Source of Truth)	其他引用位置
申請狀態機轉換規則	m6_aply/service.py	前端 AplyStatusFlow.tsx 僅作視覺化
RBAC 權限規則	app/permissions.py	前端 ProtectedRoute 僅作 UI 隱藏
月攤提計算公式	m3_fina/service.py 或 SQL generated column	前端 ContractForm 僅作預覽計算
帳號生命週期狀態	m2_asst/service.py 寫入 acct_status	前端 AccountForm 僅作顯示
速率限制 80% 預警閾值	m4_usge/service.py 常數	前端 UtilizationGauge 配色閾值需同步

「真實來源」原則:任何業務規則的修改,優先改後端;前端只應該反映 (顯示與輸入驗證),不應該獨立判斷業務邏輯。這確保系統一致性,也符合人機協作三大定律中「人有最終否決權」的精神 (前端 UI 不可繞過後端驗證)。

— 本說明書結束 —